Action Research on the DevSecOps Pipeline

Lyuben Asenov Nikolov and Adelina Plamenova Aleksieva-Petrova
Department of Computer Systems, Faculty of Computer Systems and Technologies
Technical University of Sofia
8 Kliment Ohridski blvd., 1000 Sofia, Bulgaria
{lan, aaleksieva}@tu-sofia.bg

2023 International Scientific Conference on Computer Science (COMSCI) | 979-8-3503-2525-6/23/$31.00 ©2023 IEEE | DOI: 10.1109/COMSCI59259.2023.10315920

Abstract – **In this paper, we explore the symbiotic relationship between automation and human expertise, highlighting how DevSecOps methodologies facilitate early detection and mitigation of vulnerabilities, thus fortifying the overall software security posture. By examining the cultural shifts required to embrace DevSecOps within organizations, we shed light on the collaborative and agile mindset necessary for successful implementation. Moreover, this paper elucidates the advancements in tooling and technologies that have accelerated the adoption of DevSecOps, including containerization, orchestration, and cloud-native architectures. We also explore the challenges and limitations practitioners face when adopting DevSecOps practices, encompassing threat modeling, secure coding practices, and regulatory compliance. The paper establishes DevSecOps as a dynamic and evolving discipline at the intersection of development, security, and operations, shaping the future of software security by embracing synergy and resilience.**

Keywords – **DevSecOps, DAST, SAST, Threat Modeling.**

I. Introduction

Transitioning to remote work has become the "new norm" during the pandemic. While it has allowed businesses to maintain operations, it has also exposed them to heightened cyber threats. Since the onset of the pandemic, emerging cyber threats have surged, resulting in a 600% increase in cybercrime [1]. Cybercriminals exploit various channels to launch attacks, ranging from traditional email phishing scams to sophisticated tactics like cross-site scripting (XSS) and SQL injection, aiming to pilfer sensitive data and victimize companies.

In the current landscape, cybersecurity risks are more prevalent than ever before. Unfortunately, the reality of information security is often underestimated until it's too late. The number of data breach victims due to hacking attacks surged by 210% in the third quarter of 2022 compared to the second quarter of the same year [1]. Data breach notifications escalated from 19 in 2019 to 617 in 2022. Each data security breach significantly impacts a company's business continuity cost, with the average business losing $4.35 million after every hacker attack. Moreover, in cases where remote work was a contributing factor, the average cost increased by an additional $1 million.

While one of the weakest points in an organization's security posture is the end user due to a lack of cyber security awareness training, another factor arises early in the code development cycle while applications are developed. Namely, poorly designed applications with no security mechanisms in place, at a later stage, leaves backdoors that malicious users could exploit.

In recent years, software development has witnessed an adaptation in how security is integrated into the development process. Traditional approaches to software security is often involved separate teams and processes, resulting in fragmented security measures and delayed vulnerability identification. However, the emergence of DevSecOps has revolutionized the software development landscape by emphasizing integrating security practices from the beginning of the development lifecycle and pay more attention to DevSecOps. [2].

DevSecOps (development (Dev), security (Sec), and operations (Ops)) is an extension of DevOps that is considered a means of continuous development, operations, and security. Because of security and safety concerns, companies are starting to consider DevSecOps when it comes to the application of DevOps and software security. This paper aims to study the state and practice of DevSecOps and to implement them, emphasizing collaboration and automation. This study also examines two of the main DevSecOps practices for detecting vulnerabilities in the earliest stage of the software development life cycle.

This research will follow an action research methodology in engaging with the development team from a large enterprise software organization to address real-world challenges and develop and implement a solution for DevSecOps practices. As a case study, the vulnerable Java web application “Spring Petclinic” will be used as a sample application designed to show how the Spring stack can be automated and used to build simple yet powerful database-oriented applications. The main goal is to investigate two methods for detecting vulnerabilities in the DevSecOps Jenkins pipeline environment through the SDLC (Software Development Life Cycle) - Static Application Security Testing and Dynamic Application Security Testing.

The results obtained from the detected threats will be analyzed, and in addition, in the methodology section, action research will be performed on how to build a model based on the DevSecOps approach, which can integrate software development with static and dynamic application security testing without losing the quality of the developed software.

II. Related Work

This section of the research focuses on performing an analysis of existing literature related to the research topic, specifically in the area of application security testing, namely static application security testing (SAST) and dynamic application security testing (DAST), examined as different approaches that fit into the DevSecOps pipeline.

A. SAST and DAST

Most research papers on this topic examine the characteristics of both types of methodologies sequentially to identify the differences between them and establish a common ground that their combined use provides a higher level of protection against cyber-attacks. Some scientific papers provide extremely detailed and systematic information on the characteristics of SAST and DAST such as [3] and [4].

There are also research papers examining both methodologies as a separate topic. There are examples of similar research papers focusing specifically on static application security testing [5]. They only address the static application security testing method but are presented in a larger and more comprehensive way. Other exemplary research papers are focused only on dynamic application security testing [6], which presents the dynamic software testing methodology in much more detail.

The primary distinction between DAST and SAST lies in their approaches to security testing. SAST examines the code of an application during the development phase to identify insecure code that could pose a security risk. Conversely, DAST assesses the live, running application and does not have access to the application's source code.

DAST operates as a black-box testing method, simulating an attack from an external perspective, assuming that the tester possesses no knowledge of the internal workings of the application. This approach allows DAST to identify security vulnerabilities that may not be detectable by SAST, particularly those that manifest during the program's execution.

Additional information about the characteristics, properties, and differences between the two types of software scanning methods can be found in [7]. It is clear that SAST is used for scanning source code at the earliest stage of the Software Development Lifecycle, while DAST performs a scan during the application's execution as a running application at the end of the SDLC [8]. During the SAST scanning stage developers and testers have full knowledge about the structure of the application with access to the source code while DAST is considered more like a testing approach as seen from the attacker's point view where testers don't have access to the source code of the application.

B. DevOps and DevSecOps models

Based on the main concepts of the DevSecOps model, security testing (in terms of code and the application in general) is mainly carried out in stages when developers try to integrate their solutions into the existing application. Since vulnerabilities can occur at any stage of the traditional DevOps model, many security tests are performed in the DevSecOps model, including Threat Modeling, SAST, and DAST. Both approaches are also adapted to the transition to more flexible SDLC models.

DevOps involves adopting iterative software development, automation, and using a programmable and declarative infrastructure. DevOps security issues often stem from conflicts between the different goals of developers and cybersecurity teams. The main feature of DevOps is the automation of many software testing and integration processes, which allows businesses to create and deliver new software versions quickly and seamlessly. Although both Agile and DevOps approaches consume the software development industry, security is sometimes overlooked in either strategy.

Based on the previous statement about DevOps, DevSecOps attempts to integrate security-related aspects into each phase of the DevOps pipeline model to improve the overall safety of the delivered product. DevSecOps offers an opportunity to increase the speed of the process for integrating changes in the software development life cycle while maintaining high standards for quality and security. The ideal goal is to "detect security issues (by design or application vulnerability) as quickly as possible," as argued in the DevSecOps article on Owasp.org [9].

III. DevSecOps practice and principles

Nowadays, organizations face multiple challenges when it comes down to the incorporation of DevSecOps into their environments. Based on the research from ZScaler [10], several issues and obstacles are known where organizations fail on the path of having successful DevSecOps practice:

- Addressing and mitigating vulnerabilities - According to findings from Security Boulevard [11], organizations that haven't embraced DevSecOps leave 50% of their applications consistently vulnerable to attacks, in contrast to the 22% vulnerability rate in organizations with a well-established DevSecOps approach. Typically, security testing is conducted towards the end of the development cycle, compelling developers to modify code in the later stages, incurring costly rework and project delays.
- Balancing speed and security - Striking a balance between speed and security is crucial in the realm of DevOps. DevOps emphasizes rapidity and flexibility, requiring all teams, including security, to match the pace to keep the innovation process running smoothly. Adhering to DevOps principles involves establishing a security foundation that is both agile, adaptable, and swift. Outdated security tools and processes struggle to effectively secure deployments, adversely affecting the speed of development and deployment.
- Security is considered a bottleneck - Perceived as a hindrance to progress. As reported by Gartner [11], "71% of CISOs state that their DevOps stakeholders continue to see security as an obstacle to achieving faster time-to-market." A common misconception within Dev and DevOps teams is that security impedes speed, often being seen as a bottleneck in the overall process.
- Lack of resources and knowledge gap.Insufficient resources and knowledge divide - Recent statistics reveal that 70% of organizations do not possess sufficient practical understanding of DevSecOps practices [12]. Additionally, a substantial challenge stems from limitations in staffing, tools, and budget allotments. Bridging the knowledge gap is equally challenging. Developers often lack proficiency in security and compliance—a widespread hurdle within the realm of DevSecOps. Likewise, security and operations teams frequently lack familiarity with both infrastructure and software development environments. This knowledge gap, coupled with the absence of a unified platform for knowledge dissemination, presents substantial obstacles to the effective adoption of DevSecOps.

- Roles and responsibility alignment - Aligning roles and responsibilities is a demanding task due to the dynamic nature of the DevOps environment, where teams undergo constant changes. Developers commonly assume that the security team is solely accountable for security and risk mitigation. However, in reality, the security team's role encompasses creating security policies, guiding developers and operators to grasp security requirements and optimal practices for producing secure code, and offering advisory support. The most challenging aspect of adopting DevSecOps often revolves around the people and organizational structure.

DevSecOps is an approach that aims to integrate security practices and principles into every stage of the software development lifecycle. DevSecOps promotes collaboration, automation, and a proactive approach to software security by merging development, security, and operations teams and processes. Here are key principles and practices associated with DevSecOps:

- Shift-Left Security - DevSecOps emphasizes the concept of shifting security practices and considerations to the left in the development process by integrating security activities early on, such as during requirements gathering, design, and coding phases, rather than treating security as an afterthought.
- Automation - Automation plays a crucial role in DevSecOps. It involves using tools, scripts, and frameworks to automate security checks, testing, deployment, and monitoring processes. Automated security scans, vulnerability assessments, and continuous monitoring help promptly identify and address security issues.
- Continuous Integration and Continuous Deployment (CI/CD) - DevSecOps encourages using CI/CD pipelines to facilitate continuous integration, continuous testing, and continuous deployment of software. Organizations can quickly and securely release software updates while maintaining the required security controls by automating the build, testing, and deployment processes.
- Collaboration and Communication - DevSecOps emphasizes collaboration and effective communication among development, security, and operations teams. Close collaboration ensures that security requirements, best practices, and feedback are integrated seamlessly into the development process. It also encourages shared responsibility for security among team members.
- Infrastructure as Code (IaC) - Infrastructure as Code is a practice where infrastructure components, such as servers, networks, and configurations, are defined and managed through code. Adopting IaC enables teams to version control and automate infrastructure changes, making security configurations more consistent, repeatable, and auditable.
- Security Testing - DevSecOps advocates for incorporating security testing throughout the software development lifecycle, which includes conducting regular security assessments, penetration testing, code reviews, and vulnerability scanning to identify and mitigate security vulnerabilities early on.
- Continuous Monitoring - Continuous monitoring involves actively monitoring software and infrastructure for security events and anomalies. It includes logging, real-time threat detection, and incident response. Continuous monitoring enables organizations to promptly detect security breaches, vulnerabilities, and suspicious activities.
- Culture of Security - DevSecOps aims to foster a culture of security within development teams and organizations. It involves creating awareness, training, and educating team members about security best practices. By instilling a security-focused mindset, organizations can ensure that security is considered by default in all aspects of software development.

Implementing these key principles and practices of DevSecOps helps organizations establish a robust and proactive security posture throughout the software development lifecycle. By integrating security into the development process from the start and embracing automation and collaboration, organizations can improve software security, enhance development efficiency, and deliver more secure and resilient applications.

IV. Creating and Implementing a DevSecOps Pipeline Model

The model of DevSecOps pipeline is shown in Fig. 1. That model is an actual DevSecOps model created in collaboration with one of the development teams within the Research and Development department of a software development company. Evaluating the current DevOps model and improving it by incorporating security into the pipeline is necessary.

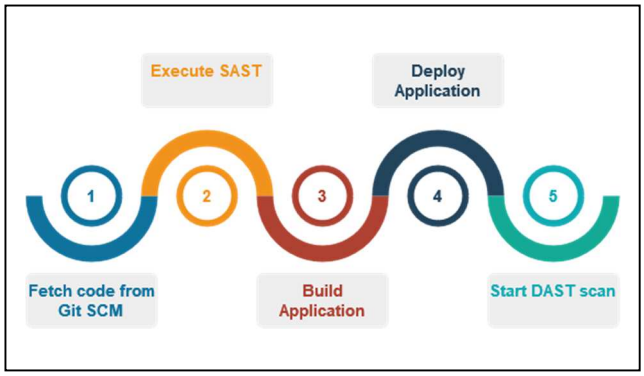

Fig. 1. DevSecOps pipeline model

The DevOps model this company was following is the same as the de facto DevOps approach that most organizations follow nowadays. Starting from the phase to push a code into a repository, followed by the Build phase, where the actual application is compiled and then deployed into stage and production environments.

As part of the case study, the following open-source technologies are included:

1) Lab environment with Jenkins server installed on a Windows Server VM residing in the Cloud Platform Azure.

2) For the lab tests, the source code of the Java web Application Spring PetClinic was cloned and pushed into a private repository on GitHub.
3) Threat modeling was performed using IriusRisk.
4) Static application security testing against the source code of the application in the earliest stage of the pipeline before the development of the application actually starts.
5) Dynamic Application security testing using Owasp ZAP after the application is deployed into production.

Inside this repository in the root directory, we have uploaded a Jenkins file that contains the full pipeline model (Fig. 2).

```
pipeline {
  agent any
  stages {
     stage('Fetch code from Git SCM) {

     }
     stage('Execute SAST') {

        #Initaite Snyk to start source code
locally
     }
     stage(Build Application') {

       #Buikd Java Source code using maven
     }
     stage(Deploy Application) {

        #Deploy and execute application
     }
     stage(Start DAST scan ') {
        #Run OWASP ZAP as a docker container
and initiate dynamic application scanning test
against the running application
     }
   }
}
```

Fig. 2. DevSecOps pipeline model as Jenkins file

So basically, in our case, we instruct Jenkins to perform the following tasks:

1) During the "Fetching code" stage, Jenkins connects to the remote private Github repository and downloads the full source code locally on the Windows VM.
2) Then followed by the "Test SAST" stage, we invoke the open-source static application security tool Snyk to scan the downloaded source code from the repository. A report is generated. During this stage, we can instruct Snyk to fail the DevOps pipeline based on the number of vulnerabilities discovered and their severity in the source code.
3) "Build Code" stage builds the Java web Spring Petclinic application using the Compiler tool Maven. The expected output file must be exported as a Jar file extension.
4) "Deploy application" stage starts the Petclinic Application in order to execute and start the application.

We engage in the final stage, "TEST DAST", to perform dynamic application security testing against the running application. During this phase, we are initiating the so-called DAST tool Owasp ZAP using a docker container in order to scan the application for vulnerabilities once it is in execution. When the scan is ready, Owasp ZAP will generate a report which we can then further analyze and check what issues were found during the scan time.

Before the start of the development process a simple threat model is performed and represent the web application's architecture on a high-level diagram. Using this approach, we can generate a list of potential threats which exist for the corresponding components. The idea at this stage of the DevSecOps methodology is to catch any vulnerabilities during the design phase of the application. Right even. Threat modeling will help us identify potential vulnerabilities in the infrastructure and in the application that may threaten an organization both from the technical and business perspective at the earliest stage of the SDLC.

In order to validate proposed model is used a sample application to demonstrate how static and dynamic application security testing can be integrated into the DevSecOps pipeline. For this purpose, the development team created a private GitHub repository was created and cloned from the official Spring Petclinic repository on GitHub. Our private GitHub repository is an exact copy of its original repository.

V. Results and Discussion

When the full DevSecOps pipeline cycle completes, Snyk and OWASP ZAP will generate a report with all the findings during the SAST and DAST stages and attach it to the Jenkins job, which holds the pipeline:

We can now start examining the reports with the threats and vulnerabilities discovered.

A. Threats and vulnerabilities from SAST using Snyk.

Based on the vulnerabilities discovered in the report from the SAST scan from Snyk, it was able to detect 27 vulnerabilities in the source code from the Spring Petclinic application (Fig. 3):

- Two Vulnerabilities with Critical Severity;
- Nine Vulnerabilities with High Severity;
- Eleven Vulnerabilities with Medium Severity;
- Five vulnerabilities with Low Severity.

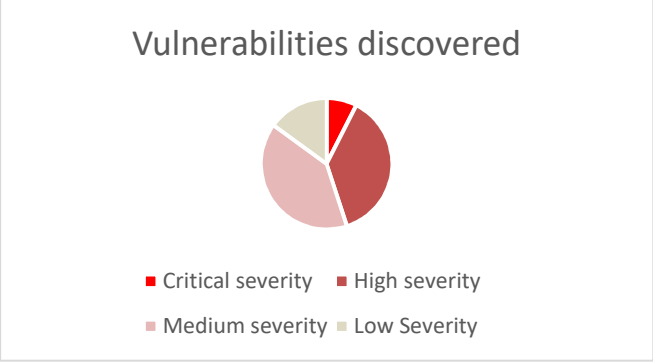

Fig. 3 Vulnerabilities chart SAST

The vulnerabilities by type are following:

- Critical severity vulnerabilities:
  - Improper access control

  - Remote Code Execution (RCE)
- High severity vulnerabilities:
  - Remote Code Execution (RCE)
  - XML External Entity (XXE) Injection
  - Denial of Service (DOS)
  - SQL injection
  - Access Restriction Bypass
  - Improper Input Validation
- Medium Severity Vulnerabilities:
  - Denial of Service (DoS)
  - Allocation of Resources Without Limits or Throttling
  - Stack-based Buffer Overflow
  - Unprotected Transport of Credentials
  - Information Exposure
  - Arbitrary Code Execution
- Low Severity Vulnerabilities:
  - Improper Handling of Case Sensitivity
  - HTTP Request Smuggling
  - Information Exposure
  - Stack-based Buffer Overflow

Although there are 27 vulnerabilities discovered, our focus is to fix and mitigate the vulnerabilities with Critical severity as Snyk's report is accurate.

B. Threats and vulnerabilities from DAST using OWASP ZAP.

Since DAST focuses on the application's runtime behavior, analyzing how it responds to different inputs, requests, and attacks (Fig. 4). It can identify vulnerabilities such as input validation errors, authentication and authorization issues, and server misconfigurations.

Name	Risk Level	Number of Instances
Source Code Disclosure - File Inclusion	High	1
Absence of Anti-CSRF Tokens	Medium	4
Anti-CSRF Tokens Check	Medium	4
Content Security Policy (CSP) Header Not Set	Medium	9
Integer Overflow Error	Medium	2
Missing Anti-clickjacking Header	Medium	9
Sub Resource Integrity Attribute Missing	Medium	10
Application Error Disclosure	Low	1
Cross-Domain JavaScript Source File Inclusion	Low	10
Information Disclosure - Debug Error Messages	Low	1
Permissions Policy Header Not Set	Low	10
X-Content-Type-Options Header Missing	Low	11
Information Disclosure - Suspicious Comments	Informational	1
Non-Storable Content	Informational	1
Storable and Cacheable Content	Informational	10
User Agent Fuzzer	Informational	168
User Controllable HTML Element Attribute (Potential XSS)	Informational	7

Fig. 4 Owasp ZAP report

Once the dynamic application security testing is complete, we aim to share our findings with the development team to understand what issues exist in the running application. Based on the results taken from the OWASP ZAP report, which was triggered against the Spring Petclinic application, we can now focus on the Vulnerabilities with Critical severity.

From this point, we have a clear overview of what we must fix. We need to work closely again with the development team to fix this threat which may expose our internal source code, allowing attackers to gain more insights about the structure of our application and the underlying OS leading to information disclosure or even a system compromise.

Once the vulnerabilities with Critical severity are fixed, we can explore the rest of the vulnerabilities and confirm if they pose a real threat to the application or if they can be considered a false positive. Dynamic Application Security Testing (DAST) tools are used by enterprises to identify and address security vulnerabilities in their web applications. These tools scan the applications in their running state to discover vulnerabilities that could potentially be exploited by attackers. Such enterprise tools should be considered as advanced dynamic application security testing solutions such as Accunetix, Veracode DAST, IBM Security AppScan, BurpSuite Professional and many more.

C. Countermeasures

It's important to implement multiple layers of defense to protect web applications from various security threats. It is really important to regularly assess and update the security measures to adapt to emerging threats and new attack vectors. Here are countermeasures and best practices to consider:

- Secure Coding Practices: Follow secure coding guidelines and best practices, such as input validation, output encoding, proper error handling, and secure session management. Avoid common coding pitfalls, like SQL injection, cross-site scripting (XSS), and cross-site request forgery (CSRF).
- Web Application Firewall (WAF): Implement a WAF to filter and monitor incoming and outgoing web traffic. A WAF can help detect and block malicious requests, including common attacks like SQL injection and XSS.
- Secure Authentication and Authorization: Implement strong authentication mechanisms, including password hashing, multi-factor authentication (MFA), and session management. Enforce appropriate authorization controls to ensure that users only have access to the resources they are authorized to use.
- Regular Security Patching: Keep all software components, including the web server, application framework, and third-party libraries, up to date with the latest security patches. Regularly apply security updates and fixes to mitigate known vulnerabilities.
- Access Control and Privilege Management: Implement the principle of least privilege, granting users and components only the necessary permissions and access rights. Employ proper access controls at different levels, including user roles, file permissions, and database privileges.
- Regular Security Testing: Conduct regular security testing, including SAST (Static Application Security Testing), DAST (Dynamic Application Security Testing), and manual security assessments. Test for vulnerabilities, misconfigurations, and security weaknesses, and address the findings promptly.

- Data Encryption: Encrypt sensitive data both at rest and in transit. Use SSL/TLS protocols for secure communication and implement appropriate encryption mechanisms for storing sensitive data, such as passwords or personal information.
- Logging and Monitoring: Implement robust logging mechanisms to track and monitor activities within the web application. Log relevant security events, error messages, and user activities to detect and investigate potential security incidents.

A relatively new security practice about protecting web applications that could be integrated with the Jenkins DevSecOps pipeline is also worth mentioning– Runtime Application Security Protection (RASP). RASP is a security technology that is embedded within the application runtime environment. It monitors the application's behavior and applies protection mechanisms in real time based on runtime data and analysis. It is a technology that acts similarly to Web Application Firewall, but the only difference is that it is embedded directly at the source code level, which is more effective in catching malicious activities against the application. RASP provides continuous monitoring and protection during the actual execution of the application. It can detect and block attacks, such as SQL injection, XSS, and code injection, by dynamically analyzing runtime data and applying security controls.

In summary, RASP provides runtime protection and immediate response to security threats, leveraging application context and behavior analysis. It is effective against emerging and unknown attacks.

VI. Conclusion

By combining SAST, DAST, and threat modeling, organizations can establish a comprehensive security approach that addresses vulnerabilities at multiple levels. The proactive nature of SAST and threat modeling enables early detection and remediation of potential security weaknesses, reducing the likelihood of successful attacks. DAST provides real-world simulation and uncovers vulnerabilities that may not be evident through static analysis alone. Integrating these techniques into the software development lifecycle promotes continuous improvement of security practices and enhances overall system resilience.

While DevOps primarily focuses on improving collaboration and efficiency in software delivery, DevSecOps adds the crucial aspect of integrating security practices into the development process. DevSecOps recognizes the importance of addressing security concerns proactively to mitigate risks and ensure the delivery of secure and resilient software in every stage of the software development lifecycle.

It's important to note that DevOps and DevSecOps are not mutually exclusive, in fact, they can complement each other. Organizations can adopt DevOps principles and practices and then incorporate security considerations to transition towards a DevSecOps approach.

Ultimately, the choice between DevOps and DevSecOps depends on an organization's specific needs, risk tolerance, and the criticality of security in their software delivery process.

Acknowledgment

The research reported here was funded by the project "Methods and tools for securing web applications", funded with contract №: 232PD0016-09 by Technical University of Sofia.

References

[1] "The 21 Latest Emerging Cyber Threats to Avoid", https://www.aura.com/ learn/emerging-cyber-threats, (accessed May 24, 2023).

[2] Mao, Runfeng, et al. "Preliminary findings about devsecops from grey literature." 2020 IEEE 20th international conference on software quality, reliability and security (QRS). IEEE, 2020.

[3] J. Yang, L. Tan, J. Peyton and K. A Duer, "Towards Better Utilizing Static Application Security Testing," 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), Montreal, QC, Canada, 2019, pp. 51-60, doi: 10.1109/ICSE-SEIP.2019.00014.

[4] T. Rangnau, R. v. Buijtenen, F. Fransen and F. Turkmen, "Continuous Security Testing: A Case Study on Integrating Dynamic Security Testing Tools in CI/CD Pipelines," 2020 IEEE 24th International Enterprise Distributed Object Computing Conference (EDOC), Eindhoven, Netherlands, 2020, pp. 145-154, doi: 10.1109/EDOC49727.2020.00026.

[5] Oyetoyan, Tosin Daniel, et al. "Myths and facts about static application security testing tools: an action research at Telenor digital." Agile Processes in Software Engineering and Extreme Programming: 19th International Conference, XP 2018, Porto, Portugal, May 21–25, 2018, Proceedings 19. Springer International Publishing, 2018.

[6] M. R. Stytz and S. B. Banks, "Dynamic software security testing," in IEEE Security & Privacy, vol. 4, no. 3, pp. 77-79, May-June 2006, doi: 10.1109/MSP.2006.64.

[7] Sharma, Manish. "Review of the Benefits of DAST (Dynamic Application Security Testing) Versus SAST." INTERNATIONAL JOURNAL OF MANAGEMENT AND ENGINEERING RESEARCH 1.1 (2021): 05-08.

[8] Dencheva, Lyubka. Comparative analysis of Static application security testing (SAST) and Dynamic application security testing (DAST) by using open-source web application penetration testing tools. Diss. Dublin, National College of Ireland, 2022.

[9] "OWASP DevSecOps Guideline - v-0.2", https://owasp.org/www-project-devsecops-guideline/latest/,(accessed May 26, 2023).

[10] "The Top Challenges Faced by Organizations Implementing DevSecOps",https://www.zscaler.com/blogs/product-insights/top-challenges-faced-organizations-implementing-devsecops, (accessed May 26, 2023).

[11] "20 Statistics That Today's DevSecOps Teams Should Know",https://securityboulevard.com/2021/05/20-statistics-that-todays-devsecops-teams-should-know/, (accessed May 26, 2023)

[12] "Developers lack skills needed for secure DevOps, survey shows",https://www.computerweekly.com/news/450424614/Developers-lack-skills-needed-for-secure-DevOps-survey-shows, (Accessed May 26, 2023)